Comparación experimental del rendimiento de Bubble Sort y Merge Sort bajo distintos escenarios de entrada

Experimental Comparison of Bubble Sort and Merge Sort Performance under Different Input Scenarios

Omar Alejandro Flores Anahua
Escuela Profesional de Ingeniería en Informática y Sistemas,
Universidad Nacional Jorge Basadre Grohmann, Tacna, Perú
ofloresa@unjbg.edu.pe

Resumen: *El presente estudio tiene como objetivo comparar experimentalmente el rendimiento de los algoritmos Bubble Sort y Merge Sort bajo distintos escenarios de entrada. Para ello, se propone implementar ambas técnicas de ordenamiento en C++ y evaluar su comportamiento con diferentes tamaños y patrones de datos. Se consideran como métricas el tiempo de ejecución, el número de comparaciones, los intercambios o movimientos de elementos y la profundidad máxima de recursión cuando corresponda. Los resultados permitirán analizar las diferencias entre el comportamiento experimental y la complejidad teórica de cada algoritmo.*

Palabras clave: algoritmos de ordenamiento, Bubble Sort, Merge Sort, rendimiento, análisis experimental.

Abstract: *This study aims to experimentally compare the performance of Bubble Sort and Merge Sort under different input scenarios. Both sorting algorithms are planned to be implemented in C++ and evaluated using different input sizes and data patterns. The metrics include execution time, number of comparisons, element swaps or movements, and maximum recursion depth where applicable. The results will make it possible to analyze the differences between the experimental behavior and the theoretical complexity of each algorithm.*

Keywords: sorting algorithms, Bubble Sort, Merge Sort, performance, experimental analysis.

1 Introducción

Los algoritmos de ordenamiento son procedimientos fundamentales en el procesamiento de datos, debido a que permiten organizar elementos de acuerdo con un criterio determinado. Su utilización facilita la búsqueda, el análisis y la presentación de información en diferentes aplicaciones informáticas. Sin embargo, los algoritmos que resuelven el mismo problema pueden presentar diferencias importantes en su tiempo de ejecución, uso de memoria y comportamiento frente a distintos tipos de entrada.

Entre los algoritmos de ordenamiento se encuentran Bubble Sort y Merge Sort. Bubble Sort compara elementos adyacentes y los intercambia cuando se encuentran en un orden incorrecto. Su implementación es sencilla, aunque su complejidad temporal promedio y en el peor caso es cuadrática. Por otro lado, Merge Sort divide los datos en partes más pequeñas, ordena cada una y combina los resultados. Este procedimiento presenta una complejidad temporal de $O(n \log n)$ en sus casos mejor, promedio y peor, aunque necesita memoria auxiliar adicional para la combinación.

El rendimiento de estos algoritmos puede variar según la cantidad de elementos y el patrón de entrada utilizado. Una lista que ya se encuentra ordenada puede favorecer a una implementación optimizada de Bubble Sort, mientras que una lista desordenada de gran tamaño puede evidenciar las diferencias entre las estrategias de ambos algoritmos. Por esta razón, resulta conveniente estudiar su comportamiento bajo diferentes condiciones experimentales.

El objetivo de este estudio es comparar experimentalmente Bubble Sort y Merge Sort considerando distintos tamaños y patrones de entrada. Para ello, se propone registrar el tiempo de ejecución, el número de comparaciones, los intercambios o movimientos de elementos y la profundidad máxima de recursión cuando corresponda.

Como expectativa descriptiva, se plantea que Bubble Sort presentará un crecimiento del tiempo de ejecución más pronunciado en entradas grandes y desordenadas, mientras que Merge Sort mostrará un crecimiento compatible con su complejidad $O(n \log n)$. Estas expectativas serán contrastadas con los datos obtenidos durante los experimentos.

2 Metodología experimental

2.1 Selección de los algoritmos

Para realizar la comparación experimental se seleccionaron Bubble Sort y Merge Sort, debido a que utilizan estrategias diferentes para resolver el problema del ordenamiento. Bubble Sort se basa en comparaciones e intercambios entre elementos adyacentes, mientras que Merge Sort emplea la estrategia de divide y vencerás. Esta selección permite estudiar las diferencias entre un algoritmo de implementación sencilla y otro que divide recursivamente el problema en partes más pequeñas.

2.1.1 Bubble Sort

Se utilizará una variante optimizada de Bubble Sort que incorpora una condición de terminación anticipada. Después de cada pasada se comprobará si se realizaron intercambios. Si no se produce ninguno, se considerará que la lista ya está ordenada y el algoritmo finalizará sin realizar pasadas innecesarias.

La variante elegida tiene una complejidad temporal de $O(n)$ en el mejor caso, cuando la entrada ya está ordenada, y de $O(n^2)$ en los casos promedio y peor. Su memoria auxiliar es $O(1)$, debido a que el procedimiento ordena los elementos en el arreglo original sin necesitar una estructura adicional proporcional al tamaño de la entrada.

La facilidad de implementación de Bubble Sort permite utilizarlo como referencia para estudiar el efecto del patrón de entrada sobre el rendimiento. No obstante, su número de comparaciones e intercambios puede aumentar considerablemente cuando los datos se encuentran desordenados.

2.1.2 Merge Sort

Merge Sort se implementará mediante una variante recursiva basada en la estrategia de divide y vencerás. El procedimiento dividirá el arreglo en dos partes, ordenará recursivamente cada una y combinará ambas partes en una secuencia ordenada.

Su complejidad temporal es $O(n \log n)$ en los casos mejor, promedio y peor. En la implementación prevista se utilizará memoria auxiliar de orden $O(n)$ para combinar los elementos. La profundidad máxima de recursión será otra característica considerada durante la evaluación experimental.

Aunque Merge Sort requiere una imple-

mentación más elaborada que Bubble Sort, su estructura permite abordar entradas grandes con un crecimiento temporal teórico menor. La utilización de memoria adicional constituye una diferencia importante que debe tenerse en cuenta al comparar ambos algoritmos.

2.1.3 Complejidad y memoria

La Tabla 2.1.3 presenta las características teóricas de las variantes seleccionadas. Estas propiedades se utilizarán como referencia para interpretar los resultados experimentales, sin sustituir las mediciones reales.

Tabla 1. Complejidad teórica de los algoritmos

Algoritmo	Mejor	Prom.	Peor	Mem.
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

En esta tabla, la memoria corresponde al espacio auxiliar adicional requerido por las implementaciones propuestas. En Bubble Sort se considera constante, mientras que en Merge Sort es proporcional al número de elementos.

2.1.4 Estabilidad

La estabilidad es la propiedad que permite conservar el orden relativo original de los elementos que poseen la misma clave. Esta característica puede ser importante cuando se ordenan registros que contienen varios campos.

La implementación de Bubble Sort evitará intercambiar elementos con claves iguales. En Merge Sort, durante la combinación, se seleccionará primero el elemento de la mitad izquierda cuando ambas claves sean iguales. Estas decisiones permitirán preservar la estabilidad, siempre que se mantengan en el código definitivo.

2.1.5 Criterios de comparación

La comparación experimental considerará el tiempo de ejecución, el número de comparaciones, los intercambios o movimientos de elementos y la profundidad máxima de recursión cuando corresponda. También se tendrá en cuenta la complejidad temporal, la memoria auxiliar y la facilidad de implementación de cada algoritmo.

Ambos algoritmos se desarrollarán en C++ y se evaluarán con entradas equivalentes. Los contadores y procedimientos de medición se definirán de forma consistente para permitir una comparación reproducible. La configuración definitiva de las implementaciones se verificará antes de ejecutar los experimentos.